

RENANCE

FEDERAL UNIVERSITY OF TECHNOLOGY, AKURE

SEN 201 — Past Questions & Materials

by Renance

Course: SEN 201

Institution: Federal University of Technology, Akure

Sources in this pack: 5

Includes past questions and lecture notes

This study pack was compiled and restructured by Renance for SEN 201 students at Federal University of Technology, Akure. It brings together 5 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app

Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 5 — Cyssa hub200 lvi materialsAaua Sen 201java manual (incomplete).pdf

Lecture/Notes · Google Drive harvest

MODULE SIX

PROGRAMMING WITH JAVA

JAVA CHARACTER SET

Java character set implies the set of characters, symbols, notations etc. applicable to JAVA

e.g. Digit -0-9

Alphabet -A-Z (upper & lower case)

Arithmetic Expression symbols =+, -, *, / etc.

Special characters =, ^, *, % etc.

EXPRESSION IN JAVA

We have Arithmetic expression and Boolean expression which is divided into two: Relational and Logical expression.

• ARITHMETIC EXPRESSION

Addition - (+)

Subtraction - (-)

Multiplication - (*)

Division - (/)

Modulus - (% or MOD)

BOOLEAN EXPRESSION

I) Relational Expression

GT > GTEQL >=

LT < LTEQL <=

EQL = NoTEQL !=

II) Logical Expression

AND - &&

OR - ||

NOT - !

== Equivalence

JAVA CLASS DECLARATION

We have (3) class declaration in JAVA

(i) Public (ii) Private (iii) Protected

PUBLIC:

In public the program is visible to all members of the class and other external classes

PRIVATE:

In private the program is only visible to the member of that class and to friend classes.

PROTECTED:

Here the program is visible to the members of that class, friend classes and desired classes

NB: Class is also referred to as file

Example:

PUBLIC class my project

{

Public static void main (string args[])

{

COMMENTING STYLE IN JAVA

We have two commenting styles which are;

1. /* Insert your comment (one line commenting style)*/
2. // Insert your comment (two or more commenting style)

DATA TYPES & I/O STATEMENTS IN JAVA

The followings are basic data types in Java:

- INTEGER
- FLOAT
- DOUBLE
- STRING
- Others are: Byte, Short, Long, Enum etc.

I/O STATEMENTS

- Scanner Method
- GUI Method

SCANNER

- Use alongside System.in
- Use with a temporary scanner

Illustrating...

```
import java.util.Scanner;
Public class ..... //your filename replaces the .....
{
    Public static void main (String args[])
    {
        int a,b;
        Scanner getin = new Scanner (System.in);
        System.out.print("Input me: ");
        a = getin.nextInt();
        System.out.print("Input the other: ");
        b = getin.nextInt();
        System.out.printf("The summation of a and b is %d", a+b);
    }
}
```

More illustration...using Math Library for computing Mathematical functions

```
import java.util.Scanner;
import java.lang.Math.*;
Public class ..... {
    Public static void main (String args[]) {
        int a,b;
        Scanner getin = new Scanner (System.in);
        System.out.print("Input me: ");
        a = getin.nextInt();
        System.out.print("Input the other: ");
        b = getin.nextInt();
        double c = Math.pow(a,b);
        System.out.printf("The power of a to b is %f", c);
    }
}
```

GUI: Graphic User Interface.**Steps to using the GUI Components for I/O functions****LIBRARY**

javax.swing

INPUT STATEMENT

JOptionPane.showInputDialog("Enter first number:");

Converts....

Integer.parseInt(d string variable);

Code Illustration...

```
import javax.swing.JOptionPane;
public class mygui {
    public static void main (String[] args) {
        int orig1, orig2;
        String adap1, adap2;
```

```

adap1=JOptionPane.showInputDialog("Enter No1: ");
adap2=JOptionPane.showInputDialog("Enter No2: ");
orig1=Integer.parseInt(adap1);
orig2=Integer.parseInt(adap2);
int tot=orig1+orig2;
JOptionPane.showMessageDialog(null, "The sumtotal is: " +tot, "A J
App",JOptionPane.ERROR_MESSAGE);
}
}

```

Lab Work VIII (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

1. Solve Lab Work I No. 1-4 using Java
2. Using Java and JOptionPane tools, code to develop an Opinion Poll System (OPS).
3. Design an Input Reaction System (IRS) using Java, e.g. if 1, prompt danger alert etc.

Coding Sheet (where this is not sufficient, please kindly print out and append)

Program ID/No: _____ Start Time: _____ Run Time: _____

Program Statement:	
1.	
2.	
3.	
4.	
5.	
6.	
7.	
8.	
9.	
10.	
11.	
12.	
13.	
14.	
15.	
16.	
17.	
18.	
19.	
20.	
21.	
22.	
23.	
24.	
25.	

MODULE SEVEN

CONTROL STATEMENTS IN JAVA

Control Statements are command statements or instructions that brought about diversion in the normal flow of program execution. Control Statements can also be programmatically defined as statements or codes that truncate the normal flow of program execution.

They function:

....either by BRANCHING,
OR
....they cause a LOOP

Implying, we have two types:

- SELECTION C.S. (BRANCHING)
&
- ITERATION C.S. (LOOPING)

Branching/Selection C. S.

A type of Control Statement that causes branching in the flow of program execution. In this control is transferred depending on the outcome of the conditional statement attached/specified.

Simply calls it a DECISION Control Statement. It decides/judges based on the parameters supplied/feed in as conditions.

Types

- **If Statement** (A one-option-one-or-none executed statement)
If (conditional statement)
 executional statement;

- **If-else Statement** (A two-options-one executed statement)
If (conditional statement)
 executional statement I;
else
 executional statement II;

If-else-if Statement (A multi-options-one executed statement)

```
If (conditional statement I)
    executional statement I;
    else if (conditional statement II)
        executional statement II;
    else
        executional statement III;
```

- **Switch-Case Statement** (A multi-options-one executed statement)
Switch(inputted/focus variable){
 Case 1: executional statement I;
 break;
 Case 2: executional statement II;
 break;

```
    .
    .
    default: ....any instructional code goes here
}
```

Iterative / Repetitive C. S.

A type of control statement that causes a particular set of codes or a segmental part of a program to be repeatedly executed. It can also be described as the type of control statement that triggers a set of actions into a continual rotational form until there occurred a counter trigger to stop them.

MODULE SEVEN

CONTROL STATEMENTS IN JAVA

Control Statements are command statements or instructions that brought about diversion in the normal flow of program execution. Control Statements can also be programmatically defined as statements or codes that truncate the normal flow of program execution.

They function:

....either by BRANCHING,
OR
....they cause a LOOP

Implying, we have two types:

- SELECTION C.S. (BRANCHING)
&
- ITERATION C.S. (LOOPING)

Branching/Selection C. S.

A type of Control Statement that causes branching in the flow of program execution. In this control is transferred depending on the outcome of the conditional statement attached/specified.

Simply calls it a DECISION Control Statement. It decides/judges based on the parameters supplied/feed in as conditions.

Types

- **If Statement** (A one-option-one-or-none executed statement)
If (conditional statement)
 executional statement;

- **If-else Statement** (A two-options-one executed statement)
If (conditional statement)
 executional statement I;
else
 executional statement II;

- **If-else-if Statement** (A multi-options-one executed statement)
If (conditional statement I)
 executional statement I;
 else if (conditional statement II)
 executional statement II;
 else
 executional statement III;

- **Switch-Case Statement** (A multi-options-one executed statement)
Switch (inputted/focus variable) {
 Case 1: executional statement I;
 break;
 Case 2: executional statement II;
 break;
 .
 .
 default:any instructional code goes here
}

Iterative / Repetitive C. S.

A type of control statement that causes a particular set of codes or a segmental part of a program to be repeatedly executed. It can also be described as the type of control statement that triggers a set of actions into a continual rotational form until there occurred a counter trigger to stop them.

- **Do-While** (A post-test looping statement)
 Do {
 executable statement(s);
 } While (conditional statement);
- **While** (A pre-test looping statement)
 While(conditional statement){
 executable statement(s);
 }

For Statement

- **For** (A range-specified looping statement)
 For(initial val; conditional val; incremental specification){
 executable statement(s);
 }

We have

- **NESTED FOR** (a FOR statement embedded in another FOR statement)
 For(int i=1,i<11,i++)
 {
 For(int j=1,j<5,j++)
 {
 }
 }
- **ENHANCED FOR** (a FOR statement used for accessing and manipulating array elements)
 For(declaration: array name)
 Declaration talks about the loop variable and its type.
 Array name implies the array in use.

Code Illustration

```
public class ..... {
    public static void main(String[] args) {
        int[] myscores = {65, 75, 81, 90, 76};
        System.out.print("Outputting my scores in this semester: ");
        for(int i: myscores){
            System.out.printf("\n%d", i);
        }
    }
}
```

Lab Work IX (*All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist*)

- Write a Java program that accepts value for an integer m, detects and decides whether the variable is a zero, a positive number or a negative number.
- Write a Java program that captures values for two integer variables num1 and num2, presents operation options that can be performed on them.
- Solve Lab Work II, III, IV, V, and VI using Java
- Get to your bank, understudy the typical components of an ATM and how it works (e.g. withdrawal, balance, recharge, deposit, pay bill etc.) and using Java, develop such an ATM using Switch-Case that will prevent/stop you from henceforth going to bank to transact, but to use your own built app.
- Do you have a DSTV at home/office (you should), else browse to study how DSTV Subscription payment options are accessed and transactions done. After your survey and study, come up with a Java designed DSTV Subscription System using Switch-Case Statement.

MODULE EIGHT

ARRAY PROGRAMMING IN JAVA

ARRAY

- DEFINITION
- DIVISIONS
- DECLARATIONS
- DEMONSTRATIONS

DEFINITIONS

An array is a collection of data element, captured with the same name (storage location name) and having the same data type.

The nature of data stored in an array depicts the type of array been created (i.e. numeric array, string array etc.)

DIVISIONS

We have two divisions of array:

- i. One Dimensional Array OR Linear Array
- ii. Multidimensional Array OR Matrix

LINEAR ARRAY

An array that consist of a one-columnar set of data.

DECLARING AN ARRAY

Declaration with Initialization

data type arrayname [empty size of the array]={d elements};

e.g.

```
int myscores[] = {6,3,4,2,1,7,8};
```

illustration...

```
package lineararray1;    //common to Netbeans IDE
import java.util.*;
public class Lineararray1 {
    public static void main(String[] args) {
        int mylist[] = {9,3,4,7,6};
        for(int i=0;i<5;i++)
        {
            System.out.printf("\nElement %d: %d", i+1, mylist[i]);
        }
    }
}
```

...output

```
Element 1: 9
Element 2: 3
Element 3: 4
Element 4: 7
Element 5: 6BUILD SUCCESSFUL (total time: 0 seconds)
```

Modify the program in Illustration by computing the sum total of the array elements –

illustration...

```
package lineararray1;
import java.util.*;
public class Lineararray1 {
```



```

public static void main(String[] args) {
    int mylist[] = {9,3,4,7,6};
    int sum = 0;

    for(int i=0; i<5; i++)
    {
        sum = sum + mylist[i];
    }
    System.out.printf("The sum total of the Array Elements is: %d\n", sum);
}
}

```

...output

The sum total of the Array Elements is: 29
BUILD SUCCESSFUL (total time: 0 seconds)

DECLARATION without INITIALIZATION

To declare a linear array in Java, we have
data type arrayname [size of the array];

e.g.

```
int myscores[10];
```

OR (for most version):

```
int myscores[] = new int[10];
```

To capture data into array, note that, ARRAY INDEX starts from 0 and not 1.
For instance, to input the above array specification, you will have:

```

package lineararray2;
import java.util.*;
public class Lineararray2 {
    public static void main(String[] args) {
        int myscores[] = new int[10];
        Scanner myscan = new Scanner(System.in);

        //capture the values now in loop

        for(int i=0; i<10; i++)
        {
            System.out.printf("Enter Element %d : ", i+1);
            myscores[i] = myscan.nextInt();
        }
    }
}

```

The output looks thus:

run:

```

Enter Element 1 :45
Enter Element 2 :67
Enter Element 3 :54
Enter Element 4 :23
Enter Element 5 :21
Enter Element 6 :12
Enter Element 7 :67
Enter Element 8 :87

```

Enter Element 9 :43
 Enter Element 10 :87
BUILD SUCCESSFUL (total time: 28 seconds)

Similar problem but with n number of elements

```
package lineararray3;
import java.util.*;
public class Lineararray3 {
    public static void main(String[] args) {
        int n;
        Scanner myscan = new Scanner (System.in);
        //capture the n value first
        System.out.print("Specify the n value:");
        n = myscan.nextInt();
        //now, define the array
        int myscores[] = new int[n];
        //capture the values now in loop
        for(int i=0; i<n; i++)
        {
            System.out.printf ("Enter Element %d :", i+1);
            myscores[i] = myscan.nextInt();
        }
    }
}
```

The output looks thus:

run:
 Specify the n value:6
 Enter Element 1 :45
 Enter Element 2 :32
 Enter Element 3 :76
 Enter Element 4 :98
 Enter Element 5 :32
 Enter Element 6 :41
BUILD SUCCESSFUL (total time: 16 seconds)

ARITHMETIC AND LOGICAL OPERATIONS ON ARRAY

Some of the arithmetic operations that can be performed on ARRAY include but not limited to:

- Summation
- Multiplication
- Comparison
- Such problems include:
 - Largest element in an array dataset
 - Summation or aggregate of an array data set
 - Sorting of array dataset

Code Examples on LINEAR ARRAY

Summation of two numeric linear arrays with 5 elements

```
package lineararray4;
import java.util.*;
public class Lineararray4 {
    public static void main(String[] args) {
        int n;
```

```

Scanner myscan = new Scanner (System.in);
//capture the n value first
System.out.print("Specify the n value:");
n = myscan.nextInt();
//now, define the array
int array1[] = new int[n];
int array2[] = new int[n];
int array3[] = new int[n];
//capture the values now in loop
System.out.print("...capturing for both Array 1&2 and Summing\n");
for(int i=0; i<n; i++)
{
    System.out.printf("\nEnter Element %d for Array 1:", i+1);
    array1[i] = myscan.nextInt();
    System.out.printf("\nEnter Element %d for Array 2:", i+1);
    array2[i] = myscan.nextInt();
    array3[i]=array1[i] + array2[i];
}
System.out.print("...outputting the new born Array (Array3)\n");
for(int i=0; i<n; i++)
{
    System.out.printf("Element %d : %d\n", i+1, array3[i]);
}
}
}

```

The output looks thus:

run:

Specify the n value:3

...capturing for both Array 1&2 and Summing

Enter Element 1 for Array 1:4

Enter Element 1 for Array 2:2

Enter Element 2 for Array 1:5

Enter Element 2 for Array 2:2

Enter Element 3 for Array 1:3

Enter Element 3 for Array 2:4

...outputting the new born Array (Array3)

Element 1 : 6

Element 2 : 7

Element 3 : 7

BUILD SUCCESSFUL (total time: 14 seconds)

Real Life Application – Selection of Applicants for Vacant Positions

This very problem was solved using C++ in Part 1 of this manual. Reference the page containing the codes in C++ and convert to Java.

Show the codes and output to a friend and also in turn check his/hers.

Lab Work X (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

1. Given the underlisted initialized original array elements (values), code in Java to output the values in your output.
NB: (1) The array size was not given. (2) The elements are to be inputted as specified. (3) The output is to be in the reverse order of the inputs
myvalues = {8, 5, 3, 5, 7, 2, 3, 4, 5, 6, 1, 17, 11, 19}
2. Given
mybox = {4, 3, 8, 5, 7, 2},
Code to sort and output this array elements in DESCENDING ORDER (i.e. from highest to lowest).
3. Fabonacii is a number series in which the 3rd number is generated from the addition of the two previous number which are usually given as 0 and 1.
How can you solve this for a particular series of number (e.g. 20 series) using FOR-LOOP and using ARRAY?

Coding Sheet (where this is not sufficient, please kindly print out and append)

Program ID/No: _____ **Start Time:** _____ **Run Time:** _____

Program Statement:

1.	
2.	
3.	
4.	
5.	
6.	
7.	
8.	
9.	
10.	
11.	
12.	
13.	
14.	
15.	
16.	
17.	
18.	
19.	
20.	
21.	
22.	
23.	
24.	
25.	

MATRIX DECLARATION

Matrix is declared thus:

`int mymatrix[3][4]`, meaning a matrix containing 3 rows of elements and 4 columns of elements

e.g.

6	7	9	5
5	4	3	9
1	4	5	8

INITIALIZATION

If initialization, the format is:

`mymatrix = {2,5,6,8,4},{9,0,3,2,1},{8,9,3,4,7}`

Which implies a matrix of 3 rows and 5 columns

e.g.

2	5	6	8	4
9	0	3	2	1
8	9	3	4	7

Programming Illustration – Inputting and Outputting Matrix (e.g. Row 3, Column 4)

```
package lineararray2;
import java.util.*;
```

```
public class Lineararray2 {
    public static void main(String[] args) {
        int mymatrix[][] = new int[3][4];
        Scanner myscan = new Scanner(System.in);
```

```
//capture the values now in loop
```

```
for(int i=0; i<3; i++)
{
    for(int j=0; j<4; j++)
    {
        System.out.printf("Enter Element in Row %d Column %d :", i+1, j+1);
        mymatrix[i][j] = myscan.nextInt();
    }
}
```

```
//to output the matrix
```

```
System.out.print("These are the MATRIX ELEMENTS\n");
for(int i=0; i<3; i++)
{
    for(int j=0; j<4; j++)
    {
        System.out.printf("%n%d\t", mymatrix[i][j]);
    }
}
}
```

The output look thus:

run:

```
Enter Element in Row 1 Column 1 :4
Enter Element in Row 1 Column 2 :3
Enter Element in Row 1 Column 3 :6
```

Enter Element in Row 1 Column 4 :2
 Enter Element in Row 2 Column 1 :4
 Enter Element in Row 2 Column 2 :3
 Enter Element in Row 2 Column 3 :7
 Enter Element in Row 2 Column 4 :5
 Enter Element in Row 3 Column 1 :4
 Enter Element in Row 3 Column 2 :3
 Enter Element in Row 3 Column 3 :5
 Enter Element in Row 3 Column 4 :2
 These are the MATRIX ELEMENTS

```
4
3
6
2
4
3
7
5
4
3
5
2
BUILD SUCCESSFUL (total time: 26 seconds)
```

To put the output in form of MATRIX structure, the little and only modification will be in the output segment codes

```
//to output the matrix
System.out.print("These are the MATRIX ELEMENTS\n");
for(int i=0; i<3; i++)
{
    System.out.print("\n");
    for(int j=0; j<4; j++)
    {
        System.out.printf("%d\t", mymatrix[i][j]);
    }
}
```

Then, the output will look thus:

```
run:
Enter Element in Row 1 Column 1 :4
Enter Element in Row 1 Column 2 :6
Enter Element in Row 1 Column 3 :3
Enter Element in Row 1 Column 4 :7
Enter Element in Row 2 Column 1 :8
Enter Element in Row 2 Column 2 :2
Enter Element in Row 2 Column 3 :4
Enter Element in Row 2 Column 4 :5
Enter Element in Row 3 Column 1 :6
Enter Element in Row 3 Column 2 :8
Enter Element in Row 3 Column 3 :9
Enter Element in Row 3 Column 4 :3
```

41

These are the MATRIX ELEMENTS

4	6	3	7
8	2	4	5
6	8	9	3

BUILD SUCCESSFUL (total time: 17 seconds)

Programming Illustration – Transposing a Matrix
 With the solution given above and code illustration given under C++ for solving the Transpose of a Matrix, study well and convert to Java.
 Show your tutor the codes and the output!

Arithmetic Operations on Matrix
 With the solution given above and code illustration given under C++ for solving Arithmetic Operations on Matrix, study well and convert to Java.
 Show your tutor the codes and the output

Lab Work XI (All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist)

- Modify the matrix arithmetic operation by:
 - Coding the inputs of hismatrix and hermatrix elements separately
 - Performing subtraction operation on the two matrices to give theirdiffmatrix
 - Performing division operation on the two matrices to give theirdivmatrix
 - Computing and outputting the inverse of each matrix

2. Given

A =	2	3	4
	5	6	7
	8	9	0

and

B = [2, 3]

Compute and output:

i. $A * B$ ii. A/B iii. A^2

(...justify mathematically, where the computation is not possible)

3. Capture the below given value for Matrix A:

A =	1	5	9
	4	6	1
	3	2	7

Then, compute and output:

- row addition of Matrix A's elements

- column addition of Matrix A's elements
- summation of Matrix A's diagonal elements
- transpose of Matrix A
- summation of Matrix A's elements and elements of Transposed Matrix A

Coding Sheet (where this is not sufficient, please kindly print out and append)

Program ID/No: _____ **Start Time:** _____ **Run Time:** _____

Program Statement:	
1.	
2.	
3.	
4.	
5.	
6.	
7.	
8.	
9.	
10.	
11.	
12.	
13.	
14.	
15.	
16.	
17.	
18.	
19.	
20.	
21.	
22.	
23.	
24.	
25.	

MODULE NINE

OOP FEATURES

ENCAPSULATION: It means hiding the internal representation of an object away from an external view. This implies that encapsulation helps to prevent other users from tampering with the internal representation (compound of an object).

POLYMORPHISM: This is an OOP concept which implies creating an object from more than one form i.e. making an object to appear in different forms.

INHERITANCE: This OOP features means deriving properties from an existing class to get a new class.

ABSTRACTION: This is an OOP concept that means deriving higher concept or method from another concept or method. This is usually obtain or achieved by making use of a particular concept or method from which a new concept or method is achieved.

DATA OR INFORMATION HIDING: In this OOP concept, data or information are declared under private or protected class such that data or information therein is not accessible by a known member class.

REUSABILITY: This OOP concept implies a situation whereby classes or properties of classes or properties of classes are inherited in other words, all sub classes in a class or particular classes are made public or accessible for re-use.

IMPLEMENTING OOP FEATURE USING JAVA

1. Encapsulation
2. Inheritance
3. Polymorphism

IMPLEMENTING ENCAPSULATION: Recall that encapsulation means an object oriented programming method in which the internal representation of an object is hidden from external i.e. the properties (data and functionalities) of that object is privatized.

Example to implement encapsulation with JAVA

Public class OUR FAMILY

```
{
Public static void main(string args[])
{
OUR PROPERTIES my properties=new OUR PROPERTIES ();
myproperties.house();
}
}
```

Public class OUR PROPERTIES

```
{
Private void class house()
{
Int no of house;
No of house=25;
System.out.print("The number of house="+ no of house);
}
}
```

IMPLEMENTING INHERITANCE: Simply put, inheritance means a programming procedure whereby an object derived properties from an existing class. Inheritance is one of the fundamental capabilities of OOP language, it is a form of software reusability in which new classes are developed quickly and easily by absorbing the properties of an existing class. With inheritance, programmers save time during program development by reusing tested program code. Note again that inheritance can be said to be a programming procedure in which a class(sub class) inherits properties(data and functionalities) from another class(super class). In addition, inheritance refers to the creation of a new class by inheriting members (sub class) from an existing class(super class).

THE CONCEPT OF SUB CLASS AND SUPER CLASS

A super class is a class from which other classes inherit properties(data and functionalities) or (attributes and method). In other words, an existing class is referred to as super class, little wonder it is called base class in c++. We have 2 forms of super class

1. Direct super class
2. Indirect super class.

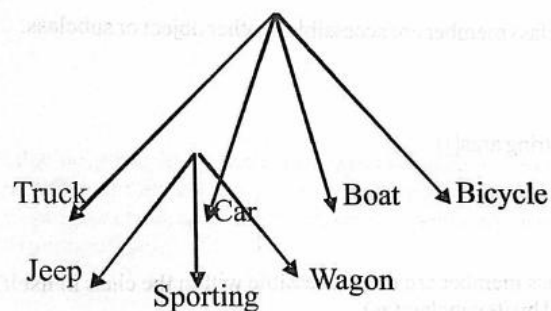
DIRECT SUPER CLASS: Is the type of class from which sub class(es) explicitly inherit properties.

INDIRECT SUPER CLASS: is the type of class from which sub class(es) implicitly inherit properties.

In other words, indirect super class is a type of class that is above the direct super class.

Example:

VEHICLE

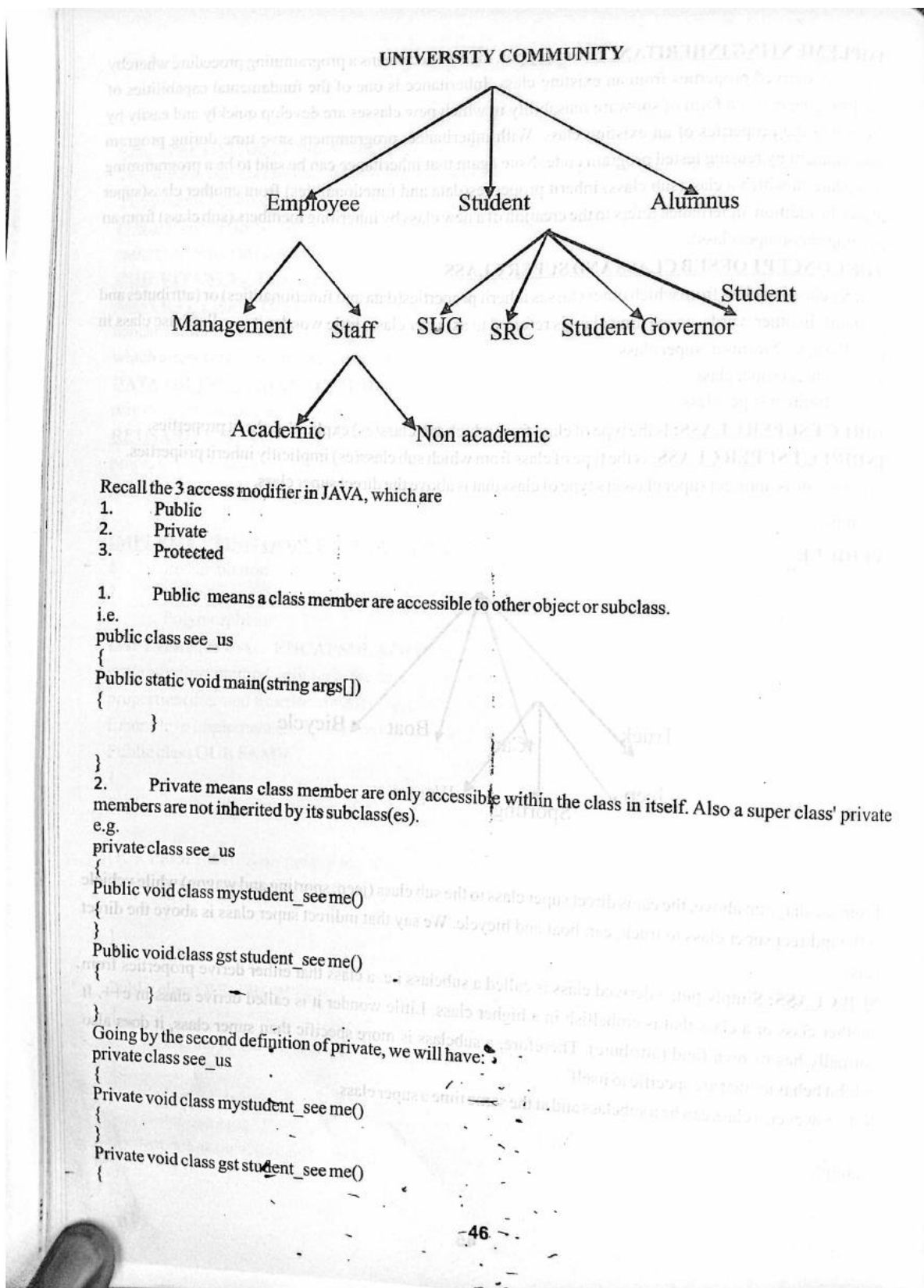


From the diagram above, the car is direct super class to the sub class (jeep, sporting and wagon) while vehicle is the indirect super class to truck, car, boat and bicycle. We say that indirect super class is above the direct class.

SUB CLASS: Simply put, a derived class is called a subclass i.e. a class that either derive properties from another class or a class that is embellish in a higher class. Little wonder it is called derive class in c++. It normally has its own field (attribute). Therefore, a subclass is more specific than super class, it does also exhibit behavior that are specific to itself.

Note however, a class can be a subclass and at the same time a super class.

Example



}
 } Using protected access modifier offers an intermediate level of access between private and public. This implies that a protected super class can be accessed by members of that super class, member of its subclass.

e.g.
 private class see_us

```
{
  public void class mystudent_see me()
{
  }
}
```

```
{
  private void class gst student_see me()
{
  }
}
```

}
 } Public class cannot inherit from private class.

Inheritance can only take place if protected access modifier is used.

e.g.

Protected class see_us

```
{
  public void class mystudent_see me()
{
  }
}
```

```
{
  private void class gst student_see me()
{
  }
}
```

POLYMORPHISM: Recall that polymorphism implies an object oriented programming method in which an object is made to appear in different forms. i.e. a way of simulating an object to behave as if it possess properties of other objects. In programming term however, we say polymorphism allows you to process (create) object that share the same super class.

ILLUSTRATION

Note that in polymorphism, objects tap the same properties but use it differently which implies inherited class will be return to the main program

IMPLEMENTING POLYMORPHISM WITH JAVA

Public class ANIMALS

```
{
  public class move()
{
  }
}
```

```
    int move level;
    move level=5;
    }return move level
```

Public class eat()

```
{
    int eatrate;
    eatrate=23;
    }return eatrate;
}
```

Public class ANIMAL KINGDOM

```
{
    Public static void main(string args[])
    {
        Float fishmove; int dogmove;
        ANIMALS fish=new ANIMALS();
        ANIMALS dog=new ANIMALS();
        ANIMALS frog=new ANIMALS();
        fish.move();
        fishmove=movelevel MOD 2;
        System.out.printf("fish moves at=",+ fishmove);
        dog.move();
        dogmove=movelevel * 3;
        System.out.printf("dog moves at=",+ dogmove);
    }
}
```

Lab Work XII (*All Lab Works are to be printed out on paper, enclosed within the manual and submitted to the Lab Technologist*)

1. Write simple but sensible JAVA programs to illustrate the following OOP features:
 - Encapsulation
 - Inheritance
 - Information Hiding
 - Polymorphism
- Abstraction

Paper 2 of 5 — Cyssa hub200 lvi materialsSen 201IMG-20260212-WA0012.jpg

Lecture/Notes · Google Drive harvest

Paper 3 of 5 — Cyssa hub200 lvi materialsSen 201IMG_20260309_124714.jpg

Lecture/Notes · Google Drive harvest

Paper 4 of 5 — Cyssa hub200 lvi materialsSen 201SEN201_Chap1.pptx

Lecture/Notes · Google Drive harvest

Paper 5 of 5 — Cyssa hub200 lvi materialsSen 201SEN201_Chap2.pptx

Lecture/Notes · Google Drive harvest
